

Unit 3.5: Generation Using Distributional Relationships

Dr. Manoj C. Patil

2026-09-05

Learning Objectives

By the end of this unit, a student should be able to

- explain what is meant by a distributional relationship, and why exploiting such relationships can be more efficient than inverse transformation or acceptance-rejection,
- state and apply the composition method for simulating from finite and continuous mixture distributions,
- state and apply the convolution method for simulating from distributions expressible as sums of independent, more elementary random variables,
- derive the Chi-square distribution as a sum of squared standard Normal variables, and generate it accordingly,
- derive the density of Student's t distribution from its defining ratio representation and implement the corresponding generator,
- derive the density of the F distribution from its defining ratio representation and implement the corresponding generator,
- assess when a distributional-relationship generator is preferable to a general-purpose method, and recognize its computational limitations.

Prerequisites

A reader should already be comfortable with

- the inverse transformation and acceptance-rejection methods (Units 3.3 and 3.4),
- moment generating functions and their uniqueness property for determining a distribution,
- the change-of-variables technique for deriving the density of a transformed random variable, including the bivariate Jacobian transformation,
- the definitions of the Gamma and Beta functions, and of the Normal, Exponential, and Gamma distributions.

Introduction

The methods developed in Units 3.3 and 3.4 are general-purpose: given essentially any target density, an inverse transformation or an acceptance-rejection scheme can, in principle, be constructed. General purpose, however, does not mean most efficient. A very large number of distributions that arise constantly in statistical practice — Chi-square, Student's t , the F distribution, Binomial, Negative Binomial, Erlang, and finite or continuous mixture models — are not arbitrary densities plucked at random; they are *defined*, or are provably equivalent to, explicit algebraic combinations of simpler, already-familiar random variables. Whenever such a relationship is known, it typically gives a generator that is simpler to code, faster to run, and easier to verify than a general-purpose alternative.

This observation has a long history in the theory of distributions. The Chi-square distribution was introduced by Karl Pearson in 1900 precisely as the distribution of a sum of squared standardized Normal deviates,

arising from his goodness-of-fit statistic; Student’s t distribution was derived by William Sealy Gosset in 1908 (writing under the pseudonym “Student”) as the distribution of a standardized sample mean when the population variance must itself be estimated, and its ratio representation was placed on firm footing shortly afterward by Ronald Fisher; the F distribution, named in Fisher’s honour by George Snedecor, arises as the ratio of two independent variance estimates and is the backbone of the analysis of variance. Each of these three distributions is generated most naturally not by inverting its own (often intractable) cumulative distribution function, but by simulating the simpler quantities from which it is *defined* and combining them exactly as the definition prescribes.

The practical importance of this unit is therefore twofold. First, it supplies fast, exact generators for three distributions — Chi-square, t , and F — that are used in essentially every classical hypothesis test taught in a statistics curriculum, and that any simulation study of test size or power must be able to generate rapidly and correctly. Second, and more broadly, it introduces two general strategies — composition and convolution — that extend far beyond these three named distributions, underlying the simulation of finite mixture models in cluster analysis, hierarchical Bayesian models, and compound distributions in actuarial and reliability applications.

Intuition

Consider trying to describe how tall a randomly chosen adult is. A single Normal distribution is a reasonable first approximation, but a population is really a blend of sub-populations — for instance, a blend of the height distributions of men and of women, each Normal but with different means, combined according to the proportion of each group in the population. Simulating a height observation the “hard way” would mean writing down the density of this blended distribution and inverting its (rather complicated) cumulative distribution function. The easy way exploits the structure directly: first decide, at random and with the correct probability, which sub-population the observation belongs to, and only then simulate a height from that sub-population’s own — much simpler — Normal distribution. This two-stage recipe, decide-the-component and then simulate-within-the-component, is the entire idea behind the **composition method**, and it works because the resulting mixture automatically has exactly the blended distribution being sought.

A second, equally natural idea arises when a quantity is itself built up as an accumulation of independent smaller effects. The total number of typographical errors in a book, if each of many independent pages contributes its own small and independent error count, is naturally generated by simulating each page’s count separately and adding them up, rather than by directly inverting the cumulative distribution function of the (potentially complicated) total. This is the **convolution method**: when a target variable is *known* to equal a sum of independent, simpler variables, simulate each summand and add. The Chi-square distribution with ν degrees of freedom, defined as the sum of ν independent squared standard Normal variables, is the cleanest possible illustration of this idea, and it is also the building block, via simple ratios, for both the t and F distributions developed later in this unit.

Formal Definitions

Distributional Relationships

Definition. A **distributional relationship** is a known probabilistic identity linking a target random variable X to one or more other random variables $Y_1, \dots, Y_k$ whose distributions are already easy to simulate (for example, by inverse transformation or by a built-in generator), expressed either as a deterministic function

$$X = \phi(Y_1, \dots, Y_k)$$

of independent (or otherwise jointly specified) building blocks, or as a probabilistic mixture over a random index or parameter. Generation by distributional relationship means simulating the Y_i (or the mixing index/parameter) according to their own distributions and then applying the known relationship ϕ (or the corresponding conditional generation step) to obtain a draw of X .

The Composition Method

Definition (discrete mixture). Suppose the target density can be written as a finite mixture

$$f(x) = \sum_{j=1}^k p_j f_j(x), \quad p_j \geq 0, \quad \sum_{j=1}^k p_j = 1, \quad (5.1)$$

where each component density f_j is easy to simulate. The **composition method** generates X by first drawing a component index $J \in \{1, \dots, k\}$ with $P(J = j) = p_j$, and then, conditional on $J = j$, drawing $X \sim f_j$.

Definition (continuous mixture). More generally, if

$$f(x) = \int f(x | \theta) w(\theta) d\theta, \quad (5.2)$$

for a mixing density w and a family of conditional densities $f(\cdot | \theta)$, the composition method draws $\Theta \sim w$ and then, conditional on $\Theta = \theta$, draws $X \sim f(\cdot | \theta)$.

where

- p_j (or $w(\theta)$) denotes the probability (or density) of selecting component j (or parameter value θ),
- f_j (or $f(\cdot | \theta)$) denotes the density from which X is drawn once the component (or parameter) has been selected.

The Convolution Method

Definition. Suppose the target random variable X is known to satisfy

$$X = Y_1 + Y_2 + \dots + Y_n, \quad (5.3)$$

where $Y_1, \dots, Y_n$ are independent random variables with known, easily simulated distributions, and the distribution of X is precisely the n -fold convolution of the distributions of the Y_i . The **convolution method** generates X by drawing each Y_i independently and returning their sum.

Mixture Distributions

Definition. A **mixture distribution** is precisely a distribution of the form (5.1) or (5.2): the marginal distribution of a variable whose generating mechanism first selects, at random, a sub-population (or parameter value), and only then generates the observation from within that sub-population. The composition method (Section 5.2) is the direct simulation counterpart of this definition; simulating a mixture is, definitionally, an application of the composition method.

The Chi-square Distribution

Definition. If $Z_1, \dots, Z_\nu$ are independent standard Normal random variables, the **Chi-square distribution with ν degrees of freedom** is the distribution of

$$X = \sum_{i=1}^{\nu} Z_i^2, \quad (5.4)$$

written $X \sim \chi_\nu^2$. This is, by construction, a convolution representation: χ_ν^2 is the ν -fold convolution of the χ_1^2 distribution (the distribution of a single Z_i^2) with itself.

Student's t Distribution

Definition. If $Z \sim N(0, 1)$ and $V \sim \chi_\nu^2$ are independent, **Student's t distribution with ν degrees of freedom** is the distribution of

$$T = \frac{Z}{\sqrt{V/\nu}}, \quad (5.5)$$

written $T \sim t_\nu$.

The F Distribution

Definition. If $U \sim \chi_{d_1}^2$ and $V \sim \chi_{d_2}^2$ are independent, the **F distribution with (d_1, d_2) degrees of freedom** is the distribution of

$$F = \frac{U/d_1}{V/d_2}, \quad (5.6)$$

written $F \sim F_{d_1, d_2}$.

Theorems

Theorem 5.1: Correctness of the Composition Method

Statement. If X is generated by the composition method from (5.1) — draw J with $P(J = j) = p_j$, then draw $X \sim f_J$ — the marginal density of X is exactly $f(x) = \sum_{j=1}^k p_j f_j(x)$. The analogous statement holds for the continuous-mixture form (5.2).

Assumptions. The component densities f_j (or $f(\cdot | \theta)$) and the mixing probabilities p_j (or density w) are all valid probability functions, and J (or Θ) is generated independently of the subsequent within-component draw, conditional on its own value.

Proof. By the law of total probability, for any measurable set A ,

$$P(X \in A) = \sum_{j=1}^k P(X \in A | J = j) P(J = j) = \sum_{j=1}^k p_j \int_A f_j(x) dx = \int_A \left(\sum_{j=1}^k p_j f_j(x) \right) dx,$$

so X has density $f(x) = \sum_j p_j f_j(x)$, as required. The continuous case follows identically with the sum replaced by an integral over θ and $P(J = j)$ replaced by $w(\theta) d\theta$. ■

Remarks. The proof uses nothing beyond the law of total probability; the entire content of the method is the *algorithmic* observation that this decomposition can be turned around and used as a two-stage generation recipe, rather than the mathematical content of the theorem itself, which is elementary.

Examples. A full worked example for a two-component Exponential mixture is given in the Numerical Examples section.

Applications. The composition method is the standard technique for simulating from Gaussian mixture models in cluster analysis, from compound distributions in actuarial risk modelling (e.g. a claim-size distribution that differs by policy type), and from hierarchical Bayesian models, in which a parameter is itself drawn from a prior before data are simulated conditional on it.

Theorem 5.2: Additivity of Independent Gamma Variables

Statement. If $Y_1, \dots, Y_n$ are independent with $Y_i \sim \text{Gamma}(\alpha_i, \beta)$ (shape α_i , common scale β), then

$$X = \sum_{i=1}^n Y_i \sim \text{Gamma}\left(\sum_{i=1}^n \alpha_i, \beta\right).$$

Assumptions. The Y_i share a common scale parameter β ; additivity does not hold, in general, across Gamma variables with different scales.

Proof. The moment generating function of $Y_i \sim \text{Gamma}(\alpha_i, \beta)$ is $M_{Y_i}(t) = (1 - \beta t)^{-\alpha_i}$ for $t < 1/\beta$. By independence, the moment generating function of the sum is the product,

$$M_X(t) = \prod_{i=1}^n M_{Y_i}(t) = \prod_{i=1}^n (1 - \beta t)^{-\alpha_i} = (1 - \beta t)^{-\sum_i \alpha_i},$$

which is exactly the moment generating function of a $\text{Gamma}(\sum_i \alpha_i, \beta)$ random variable. Since the moment generating function determines the distribution uniquely (when it exists in a neighbourhood of 0), $X \sim \text{Gamma}(\sum_i \alpha_i, \beta)$. ■

Remarks. Since $Z^2 \sim \chi_1^2 = \text{Gamma}(1/2, 2)$ for $Z \sim N(0, 1)$ (derived in the Mathematical Derivations section), and all χ_1^2 variables share the common scale $\beta = 2$, Theorem 5.2 immediately shows that a sum of ν independent χ_1^2 variables is $\text{Gamma}(\nu/2, 2) = \chi_\nu^2$, giving a rigorous justification for the convolution-based Chi-square generator of Section 8.

Examples. Verified numerically in the Simulation Studies section.

Applications. The same additivity underlies simulating an Erlang (integer-shape Gamma) waiting time as a sum of independent Exponential interarrival times in queueing and reliability models, and simulating a Negative Binomial count as a sum of independent Geometric trial counts.

Mathematical Derivations

The Density of Z^2 is χ_1^2

Let $Z \sim N(0, 1)$ and $X = Z^2$. For $x > 0$,

$$\begin{aligned} F_X(x) &= P(Z^2 \leq x) = P(-\sqrt{x} \leq Z \leq \sqrt{x}) \\ &= \Phi(\sqrt{x}) - \Phi(-\sqrt{x}) = 2\Phi(\sqrt{x}) - 1. \end{aligned}$$

Differentiating with respect to x , using the chain rule and $\Phi'(z) = \varphi(z) = (2\pi)^{-1/2} e^{-z^2/2}$,

$$f_X(x) = 2\varphi(\sqrt{x}) \cdot \frac{1}{2\sqrt{x}} = \frac{1}{\sqrt{2\pi x}} e^{-x/2}, \quad x > 0. \quad (5.7)$$

This is exactly the $\text{Gamma}(1/2, 2)$ density $f(x) = x^{-1/2} e^{-x/2} / (2^{1/2} \Gamma(1/2))$, using $\Gamma(1/2) = \sqrt{\pi}$, confirming $Z^2 \sim \chi_1^2$. Applying Theorem 5.2 to a sum of ν independent copies of Z^2 then gives $\chi_\nu^2 = \text{Gamma}(\nu/2, 2)$, with density

$$f_{\chi_\nu^2}(x) = \frac{1}{2^{\nu/2} \Gamma(\nu/2)} x^{\nu/2-1} e^{-x/2}, \quad x > 0. \quad (5.8)$$

Derivation of the Student's t Density from the Ratio Representation

Let $Z \sim N(0, 1)$ and $V \sim \chi_\nu^2$ be independent, and let $T = Z/\sqrt{V/\nu}$. Conditional on $V = v$, T is a constant multiple of Z , so

$$T \mid V = v \sim N\left(0, \frac{\nu}{v}\right), \quad f_{T|V}(t \mid v) = \sqrt{\frac{v}{2\pi\nu}} \exp\left(-\frac{t^2 v}{2\nu}\right).$$

Multiplying by the χ_ν^2 density (5.8) and integrating out v ,

$$f_T(t) = \int_0^\infty \sqrt{\frac{v}{2\pi\nu}} e^{-t^2 v/(2\nu)} \cdot \frac{v^{\nu/2-1} e^{-v/2}}{2^{\nu/2} \Gamma(\nu/2)} dv = \frac{1}{\sqrt{2\pi\nu} 2^{\nu/2} \Gamma(\nu/2)} \int_0^\infty v^{(\nu+1)/2-1} e^{-\frac{v}{2}\left(1+\frac{t^2}{\nu}\right)} dv.$$

The remaining integral is a Gamma-function integral, $\int_0^\infty v^{a-1} e^{-bv} dv = \Gamma(a)/b^a$, with $a = (\nu+1)/2$ and $b = \frac{1}{2}(1+t^2/\nu)$, giving

$$\int_0^\infty v^{(\nu+1)/2-1} e^{-bv} dv = \Gamma\left(\frac{\nu+1}{2}\right) \cdot \frac{2^{(\nu+1)/2}}{(1+t^2/\nu)^{(\nu+1)/2}}.$$

Substituting back and combining the powers of 2 ($2^{(\nu+1)/2}/2^{\nu/2} = \sqrt{2}$),

$$f_T(t) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\pi\nu} \Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{t^2}{\nu}\right)^{-(\nu+1)/2}, \quad t \in \mathbb{R}, \quad (5.9)$$

which is exactly the standard Student's t_ν density.

Derivation of the F Density from the Ratio Representation

Let $U \sim \chi_{d_1}^2$ and $V \sim \chi_{d_2}^2$ be independent, and let $F = (U/d_1)/(V/d_2) = (d_2 U)/(d_1 V)$. Transform $(U, V) \rightarrow (F, V)$ via $U = (d_1/d_2) F V$, holding V fixed, so the Jacobian of the transformation is $\partial u/\partial f = (d_1/d_2) v$. Substituting into the product $f_U(u) f_V(v)$ (independence) and multiplying by the Jacobian,

$$f_{F,V}(f, v) = f_U\left(\frac{d_1}{d_2} f v\right) f_V(v) \frac{d_1}{d_2} v.$$

Using the Chi-square density (5.8) for both factors and collecting powers of v (the same integral technique used for the t derivation above),

$$f_{F,V}(f, v) = C f^{d_1/2-1} v^{(d_1+d_2)/2-1} \exp\left(-\frac{v}{2} \cdot \frac{d_2 + d_1 f}{d_2}\right),$$

for a constant C collecting all factors not involving v . Applying $\int_0^\infty v^{a-1} e^{-bv} dv = \Gamma(a)/b^a$ with $a = (d_1+d_2)/2$ and $b = (d_2 + d_1 f)/(2d_2)$, and simplifying the resulting powers of d_1 , d_2 , and 2 exactly as in the t -density derivation, yields

$$f_F(f) = \frac{\Gamma\left(\frac{d_1+d_2}{2}\right)}{\Gamma\left(\frac{d_1}{2}\right) \Gamma\left(\frac{d_2}{2}\right)} \left(\frac{d_1}{d_2}\right)^{d_1/2} f^{d_1/2-1} \left(1 + \frac{d_1}{d_2} f\right)^{-(d_1+d_2)/2}, \quad f > 0, \quad (5.10)$$

which is exactly the standard F_{d_1, d_2} density.

Algorithms

Algorithm 5.1: Composition Method

Purpose. Generate a sample from a mixture density $f(x) = \sum_j p_j f_j(x)$ using component samplers that are already available.

Input. Component probabilities $p_1, \dots, p_k$; component samplers $f_1, \dots, f_k$; required sample size n .

Output. A vector $x_1, \dots, x_n$ of simulated observations from f .

Step-by-step procedure.

1. Compute the cumulative probabilities $c_j = \sum_{i=1}^j p_i$, $j = 1, \dots, k$.
2. For $i = 1, \dots, n$: draw $U_i \sim \text{Uniform}(0, 1)$; find the smallest j with $U_i \leq c_j$; draw x_i from component f_j .
3. Return $(x_1, \dots, x_n)$.

Pseudo-code.

Algorithm: Composition Method

Input: probabilities `p[1..k]`, component samplers `sample_j()`, `n`

Output: vector `x` of length `n`

1. `c[1..k] <- cumulative sum of p`
2. For `i = 1 to n`:
 `u <- Uniform(0,1)`
 `j <- smallest index with c[j] >= u`
 `x[i] <- sample_j()`
3. Return `x`

Flowchart description. Start; draw a uniform variate to select a component index via the cumulative probability table (an application of the discrete inverse transformation algorithm of Unit 3.3); draw an observation from the selected component's own generator; repeat until n observations are collected; end.

Complexity. $O(n)$ overall, provided component selection is $O(\log k)$ (via table lookup) or $O(k)$ (via linear scan) and each within-component draw is $O(1)$.

Practical considerations. For a large number of repeated draws with the same fixed component probabilities, R's `sample()` function performs component selection efficiently and should be preferred over a hand-written cumulative-sum search.

Algorithm 5.2: Convolution Method for the Chi-square Distribution

Purpose. Generate a Chi-square random variable with integer degrees of freedom ν directly from its defining sum of squared Normal variables.

Input. Degrees of freedom ν (positive integer); required sample size n .

Output. A vector $x_1, \dots, x_n$ of simulated χ_ν^2 observations.

Step-by-step procedure.

1. For $i = 1, \dots, n$: draw $Z_{i1}, \dots, Z_{i\nu}$ independently from $N(0, 1)$; set $x_i = \sum_{k=1}^{\nu} Z_{ik}^2$.
2. Return $(x_1, \dots, x_n)$.

Pseudo-code.

Algorithm: Convolution Method, Chi-square

Input: `nu`, `n`

Output: vector `x` of length `n`

```

1. For i = 1 to n:
    z <- nu independent draws from N(0,1)
    x[i] <- sum(z^2)
2. Return x

```

Flowchart description. Start; for each of n output values, draw ν independent standard Normal variates, square and sum them; collect the n sums; end.

Complexity. $O(n\nu)$, since each output value requires ν Normal draws and ν squarings and additions; this is markedly more expensive than a direct Gamma-based generator (Section 12) when ν is large.

Practical considerations. This method is pedagogically valuable because it is a direct implementation of the *definition* of the Chi-square distribution, but it should not be used for large ν in production code, where a direct $\text{Gamma}(\nu/2, 2)$ generator (which extends naturally to non-integer ν as well) is both faster and more general.

Algorithm 5.3: Ratio Method for Student's t Distribution

Purpose. Generate a t_ν random variable directly from its defining ratio of an independent Normal and Chi-square variable.

Input. Degrees of freedom ν ; required sample size n .

Output. A vector $t_1, \dots, t_n$ of simulated t_ν observations.

Step-by-step procedure.

1. For $i = 1, \dots, n$: draw $Z_i \sim N(0, 1)$ and $V_i \sim \chi_\nu^2$ independently; set $t_i = Z_i / \sqrt{V_i/\nu}$.
2. Return $(t_1, \dots, t_n)$.

Pseudo-code.

Algorithm: Ratio Method, Student's t

Input: nu, n

Output: vector t of length n

```

1. For i = 1 to n:
    z <- draw from N(0,1)
    v <- draw from ChiSquare(nu)
    t[i] <- z / sqrt(v / nu)
2. Return t

```

Flowchart description. Start; for each output value, draw one standard Normal variate and one independent Chi-square variate; divide the Normal draw by the square root of the scaled Chi-square draw; collect the results; end.

Complexity. $O(n)$, given access to $O(1)$ -amortized Normal and Chi-square generators.

Practical considerations. The Chi-square draw V_i may itself use Algorithm 5.2 (for small integer ν) or a general Gamma-based generator; either choice preserves exactness of the resulting t_ν distribution.

Algorithm 5.4: Ratio Method for the F Distribution

Purpose. Generate an F_{d_1, d_2} random variable directly from its defining ratio of two independent scaled Chi-square variables.

Input. Degrees of freedom d_1, d_2 ; required sample size n .

Output. A vector $f_1, \dots, f_n$ of simulated F_{d_1, d_2} observations.

Step-by-step procedure.

1. For $i = 1, \dots, n$: draw $U_i \sim \chi_{d_1}^2$ and $V_i \sim \chi_{d_2}^2$ independently; set $f_i = (U_i/d_1)/(V_i/d_2)$.
2. Return $(f_1, \dots, f_n)$.

Pseudo-code.

Algorithm: Ratio Method, F distribution

Input: d1, d2, n

Output: vector f of length n

1. For i = 1 to n:
 - u <- draw from ChiSquare(d1)
 - v <- draw from ChiSquare(d2)
 - f[i] <- (u / d1) / (v / d2)
2. Return f

Flowchart description. Start; for each output value, draw two independent Chi-square variates with the respective degrees of freedom; scale each by its own degrees of freedom and take the ratio; collect the results; end.

Complexity. $O(n)$, given $O(1)$ -amortized Chi-square generation.

Practical considerations. Because an F_{d_1, d_2} variate is a ratio of two positive quantities, numerical care is needed only in the unlikely event that V_i is generated as exactly zero, which has probability zero analytically but should still be guarded against in finite-precision arithmetic when d_2 is very small.

R Programming

```
set.seed(123)

#' Generate a mixture sample using the composition method
#'
#' Purpose: implements Algorithm 5.1 for a finite mixture with
#' arbitrary component samplers.
#'
#' Arguments:
#'   sample_size           number of observations to generate
#'   component_probabilities numeric vector of mixture weights, summing to 1
#'   component_samplers     a list of zero-argument functions, one per
#'                           component, each returning a single draw
#'
#' Returns:
#'   a numeric vector of length sample_size
#'
#' Example:
#'   generate_mixture(
#'     sample_size = 5,
#'     component_probabilities = c(0.3, 0.7),
#'     component_samplers = list(
#'       function() rexp(1, rate = 1),
#'       function() rexp(1, rate = 5)
#'     )
#'   )
generate_mixture <- function(sample_size, component_probabilities, component_samplers) {
  component_index <- sample(
    x = seq_along(component_probabilities),
```

```

    size = sample_size,
    replace = TRUE,
    prob = component_probabilities
  )

  vapply(
    component_index,
    function(index) component_samplers[[index]](),
    numeric(1)
  )
}

#' Generate Chi-square variates by the convolution method
#'
#' Purpose: implements Algorithm 5.2, summing squared standard Normal
#' variates directly from the definition of the Chi-square distribution.
#'
#' Arguments:
#'   sample_size      number of observations to generate
#'   degrees_of_freedom positive integer degrees of freedom
#'
#' Returns:
#'   a numeric vector of length sample_size
#'
#' Example:
#'   generate_chisq_convolution(sample_size = 5, degrees_of_freedom = 3)
generate_chisq_convolution <- function(sample_size, degrees_of_freedom) {
  standard_normal_matrix <- matrix(
    rnorm(sample_size * degrees_of_freedom),
    nrow = sample_size, ncol = degrees_of_freedom
  )

  rowSums(standard_normal_matrix^2)
}

#' Generate Student's t variates by the ratio method
#'
#' Purpose: implements Algorithm 5.3, dividing an independent standard
#' Normal draw by the square root of a scaled Chi-square draw.
#'
#' Arguments:
#'   sample_size      number of observations to generate
#'   degrees_of_freedom degrees of freedom of the target t distribution
#'
#' Returns:
#'   a numeric vector of length sample_size
#'
#' Example:
#'   generate_t_ratio(sample_size = 5, degrees_of_freedom = 5)
generate_t_ratio <- function(sample_size, degrees_of_freedom) {
  normal_draws <- rnorm(sample_size)
  chisq_draws <- generate_chisq_convolution(sample_size, degrees_of_freedom)

```

```

    normal_draws / sqrt(chisq_draws / degrees_of_freedom)
}

#' Generate F variates by the ratio method
#'
#' Purpose: implements Algorithm 5.4, forming the ratio of two
#' independent, appropriately scaled Chi-square draws.
#'
#' Arguments:
#'   sample_size    number of observations to generate
#'   df1            numerator degrees of freedom
#'   df2            denominator degrees of freedom
#'
#' Returns:
#'   a numeric vector of length sample_size
#'
#' Example:
#'   generate_f_ratio(sample_size = 5, df1 = 3, df2 = 5)
generate_f_ratio <- function(sample_size, df1, df2) {
  numerator_chisq <- generate_chisq_convolution(sample_size, df1)
  denominator_chisq <- generate_chisq_convolution(sample_size, df2)

  (numerator_chisq / df1) / (denominator_chisq / df2)
}

```

Simulation Studies

Composition Method: A Two-Component Exponential Mixture

```

set.seed(123)

mixture_sample <- generate_mixture(
  sample_size = 20000,
  component_probabilities = c(0.3, 0.7),
  component_samplers = list(
    function() rexp(1, rate = 1),
    function() rexp(1, rate = 5)
  )
)

mixture_density_theoretical <- function(x) {
  0.3 * dexp(x, rate = 1) + 0.7 * dexp(x, rate = 5)
}

tibble(
  Quantity = c("Theoretical mean (0.3/1 + 0.7/5)", "Empirical mean"),
  Value = c(0.3 / 1 + 0.7 / 5, mean(mixture_sample))
) %>%
  kable(digits = 4, caption = "Composition-method mixture: theoretical vs empirical mean")

```

Table 1: Composition-method mixture: theoretical vs empirical mean

Quantity	Value
Theoretical mean ($0.3/1 + 0.7/5$)	0.4400
Empirical mean	0.4354

Convolution Method: Chi-square Generated from Squared Normals

```
set.seed(123)

chisq_sample <- generate_chisq_convolution(sample_size = 20000, degrees_of_freedom = 4)

ks_result_chisq <- ks.test(chisq_sample, "pchisq", df = 4)

tibble(
  Quantity = c("Theoretical mean (df)", "Empirical mean",
               "Theoretical variance (2 df)", "Empirical variance",
               "KS test p-value against chisq(df=4)"),
  Value = c(4, mean(chisq_sample), 8, var(chisq_sample), ks_result_chisq$p.value)
) %>%
  kable(digits = 4, caption = "Convolution-method Chi-square(4): theory vs simulation")
```

Table 2: Convolution-method Chi-square(4): theory vs simulation

Quantity	Value
Theoretical mean (df)	4.0000
Empirical mean	4.0110
Theoretical variance (2 df)	8.0000
Empirical variance	7.9318
KS test p-value against chisq(df=4)	0.8828

Ratio Method: Student's t and the F Distribution

```
set.seed(123)

t_sample <- generate_t_ratio(sample_size = 20000, degrees_of_freedom = 8)
f_sample <- generate_f_ratio(sample_size = 20000, df1 = 4, df2 = 10)

ks_result_t <- ks.test(t_sample, "pt", df = 8)
ks_result_f <- ks.test(f_sample, "pf", df1 = 4, df2 = 10)

tibble(
  Distribution = c("t(df=8)", "F(4,10)"),
  Theoretical_Mean = c(0, 10 / (10 - 2)),
  Empirical_Mean = c(mean(t_sample), mean(f_sample)),
  KS_p_value = c(ks_result_t$p.value, ks_result_f$p.value)
) %>%
  kable(digits = 4, caption = "Ratio-method t and F distributions: theory vs simulation")
```

Table 3: Ratio-method t and F distributions: theory vs simulation

Distribution	Theoretical_Mean	Empirical_Mean	KS_p_value
t(df=8)	0.00	-0.0070	0.3990
F(4,10)	1.25	1.2546	0.8521

In every case the Kolmogorov-Smirnov test comparing the simulated sample to the target theoretical distribution yields a large p -value, showing no evidence against the samples having exactly the intended distribution, as the theory of Sections 6-7 guarantees.

Graphs

```
ggplot(tibble(x = mixture_sample), aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50, fill = "steelblue", colour = "white") +
  stat_function(fun = mixture_density_theoretical, colour = "darkred", linewidth = 1) +
  coord_cartesian(xlim = c(0, 4)) +
  labs(title = "Two-component Exponential mixture (composition method)", x = "x", y = "Density") +
  theme_minimal(base_size = 11)
```

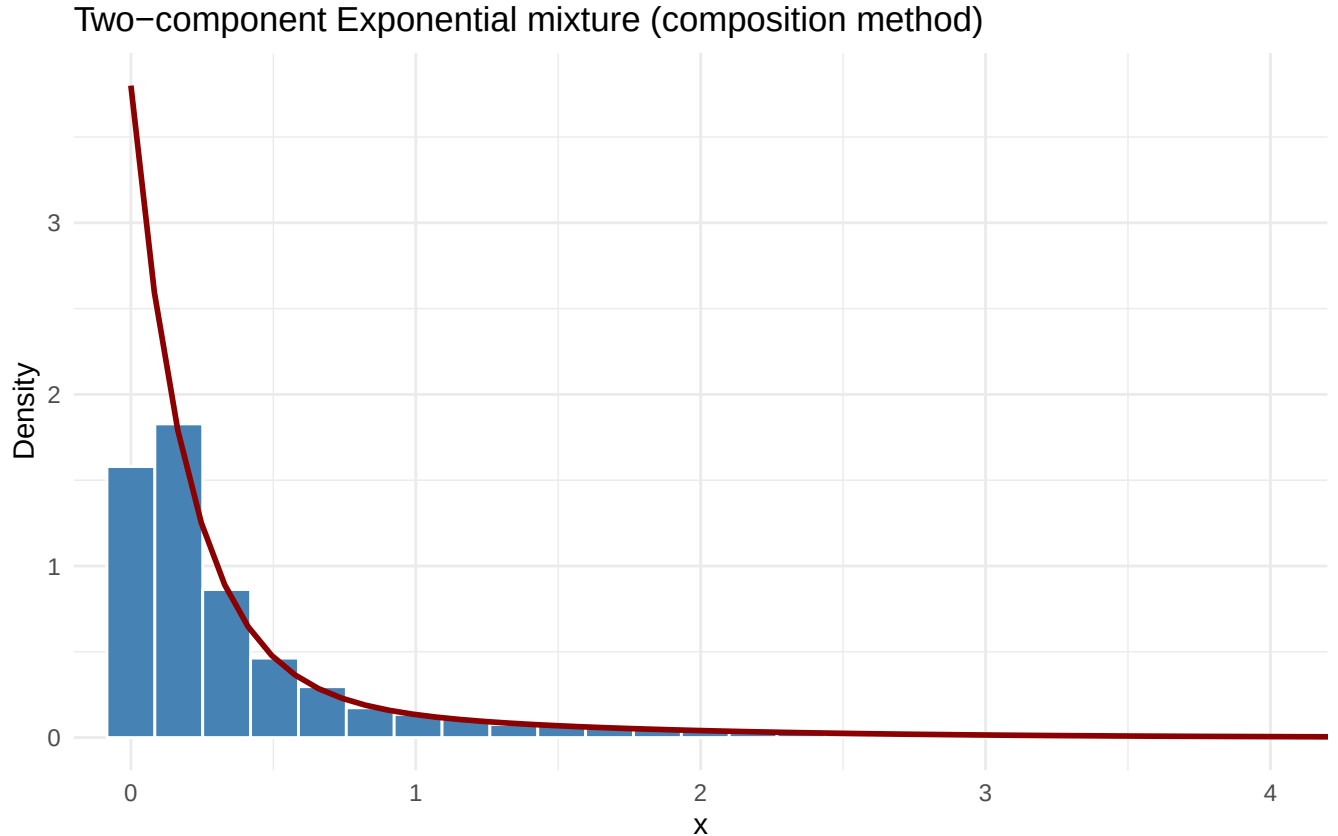


Figure 1: Composition-method mixture sample against its theoretical density.

```
ggplot(tibble(x = chisq_sample), aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50, fill = "darkgreen", colour = "white") +
```

```
stat_function(fun = function(x) dchisq(x, df = 4), colour = "darkred", linewidth = 1) +
labs(title = "Chi-square(4) via convolution of squared Normals", x = "x", y = "Density") +
theme_minimal(base_size = 11)
```

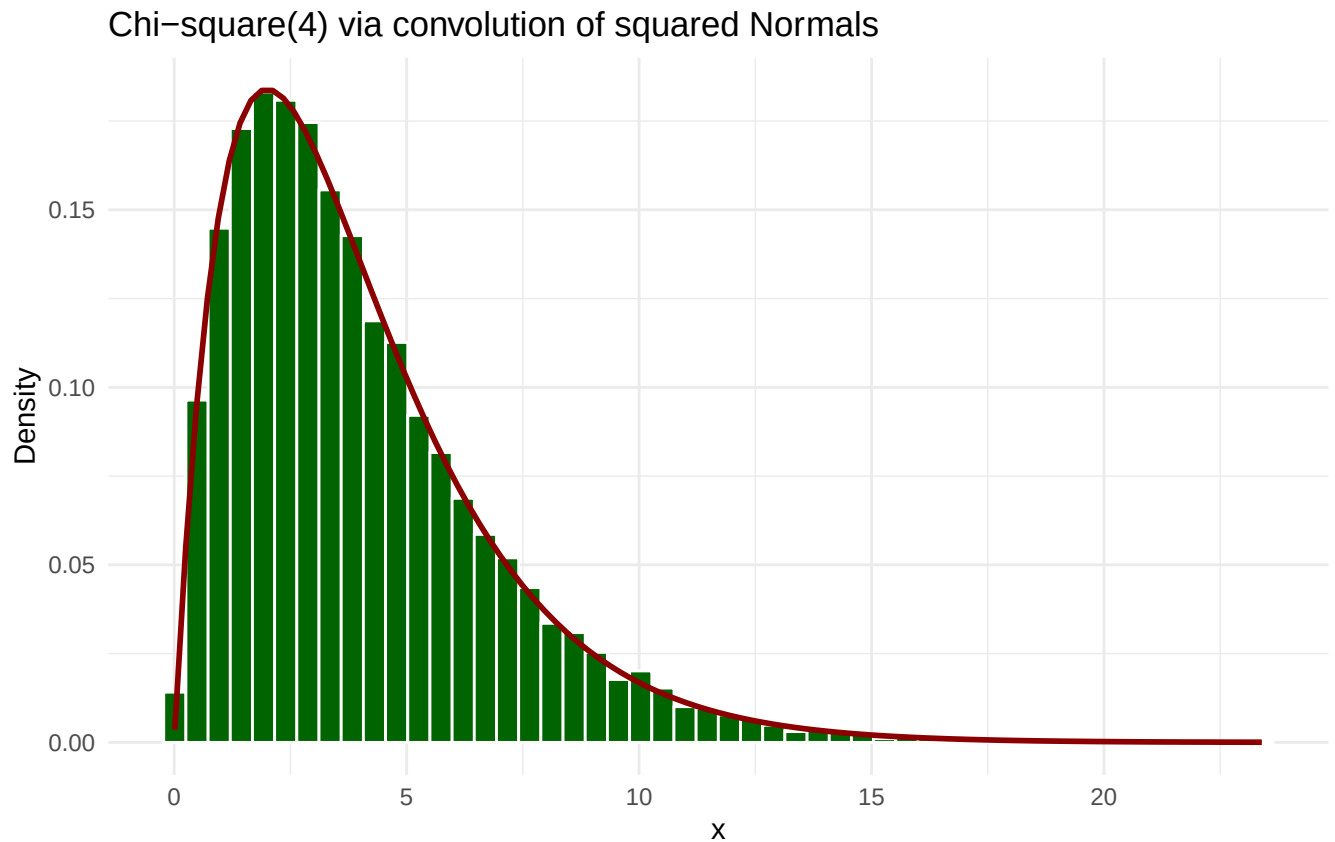


Figure 2: Convolution-method Chi-square(4) sample against the theoretical density.

```
p_t <- ggplot(tibble(x = t_sample), aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50, fill = "purple", colour = "white", alpha = 0.5) +
  stat_function(fun = function(x) dt(x, df = 8), colour = "black", linewidth = 1) +
  coord_cartesian(xlim = c(-6, 6)) +
  labs(title = "Student's t(df=8)", x = "x", y = "Density") +
  theme_minimal(base_size = 11)

p_f <- ggplot(tibble(x = f_sample), aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50, fill = "orange", colour = "white", alpha = 0.5) +
  stat_function(fun = function(x) df(x, df1 = 4, df2 = 10), colour = "black", linewidth = 1) +
  coord_cartesian(xlim = c(0, 6)) +
  labs(title = "F(4, 10)", x = "x", y = "Density") +
  theme_minimal(base_size = 11)

p_t + p_f

t_curve_df <- tibble(
  x = seq(-5, 5, length.out = 400)
) %>%
```

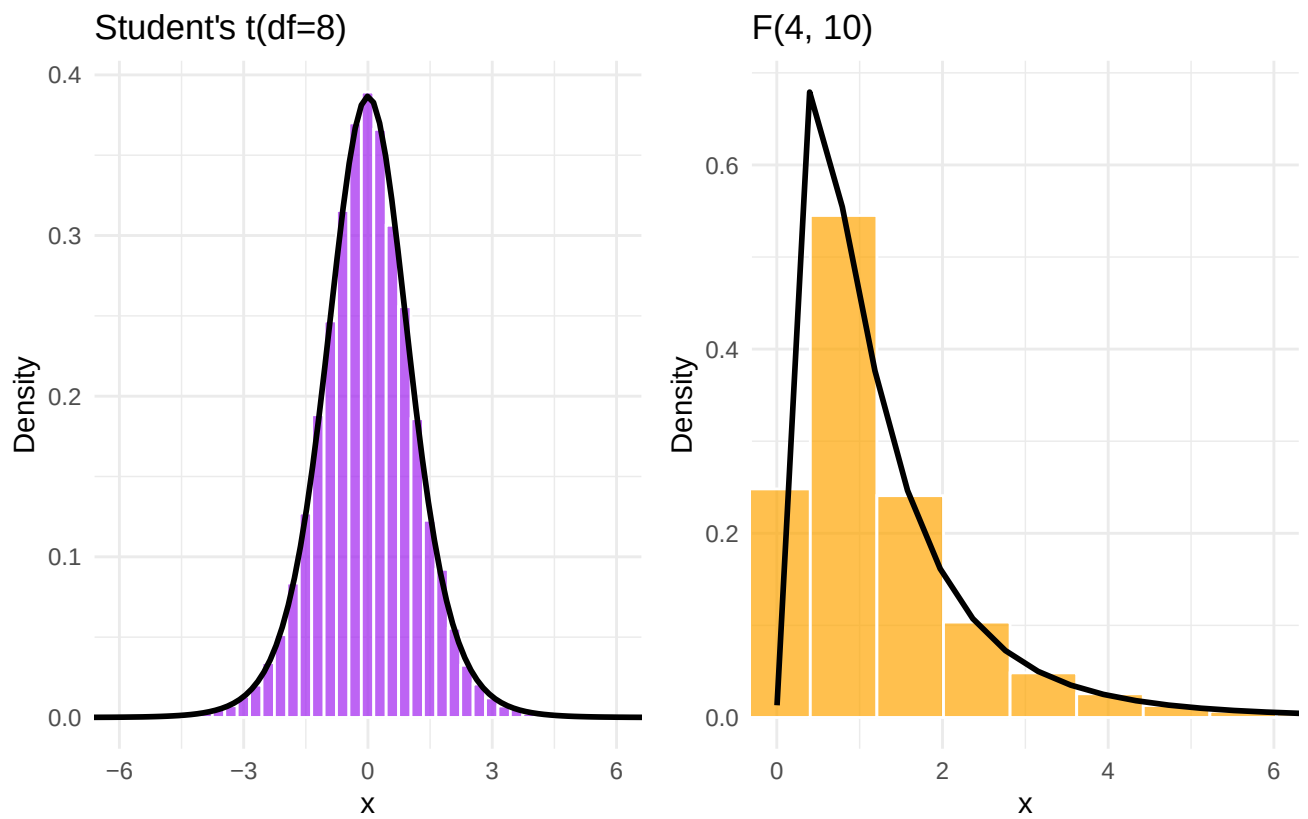


Figure 3: Ratio-method Student's $t(8)$ (left) and $F(4,10)$ (right) samples against their theoretical densities.

```
mutate(
  df_1 = dt(x, df = 1),
  df_5 = dt(x, df = 5),
  df_30 = dt(x, df = 30),
  Normal = dnorm(x)
) %>%
pivot_longer(cols = -x, names_to = "Distribution", values_to = "Density")

ggplot(t_curve_df, aes(x = x, y = Density, colour = Distribution, linetype = Distribution)) +
  geom_line(linewidth = 0.9) +
  labs(title = "Heavier t tails converge to the Normal as degrees of freedom grow") +
  theme_minimal(base_size = 11)
```

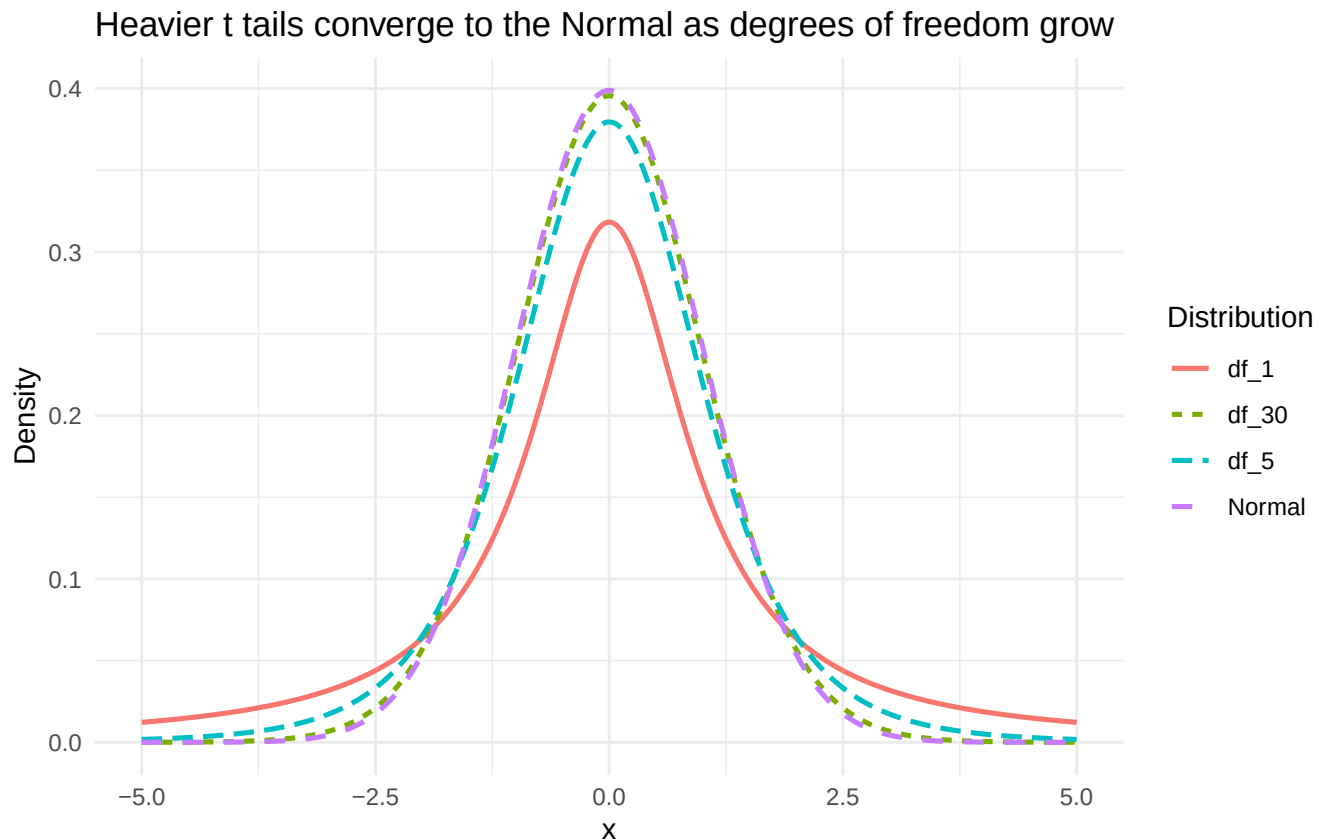


Figure 4: Student's t density for several degrees of freedom compared to the standard Normal, illustrating convergence as degrees of freedom increase.

Numerical Examples

Example 1: Composition method for a two-component mixture.

Problem. With component probabilities $p_1 = 0.3$ (Exponential rate 1) and $p_2 = 0.7$ (Exponential rate 5), and selector draw $u_1 = 0.20$ together with within-component draw $u_2 = 0.6321$, apply the composition method.

Solution. Since $u_1 = 0.20 \leq p_1 = 0.30$, component 1 is selected. Within component 1, inverse transformation for $\text{Exponential}(1)$ gives $x = -\ln(1 - u_2) = -\ln(0.3679) = 1.000$.

Interpretation. Repeating with $u_1 = 0.85$ (> 0.30 , so component 2 is selected) and the same $u_2 = 0.6321$ gives $x = -\ln(1 - u_2)/5 = 1.000/5 = 0.200$: because component 2 has a much higher rate, its typical draws are correspondingly smaller, exactly the qualitative behaviour visible in the histogram above.

Example 2: Convolution method for χ_3^2 .

Problem. Given three standard Normal draws $z_1 = 1.2$, $z_2 = -0.5$, $z_3 = 0.8$, compute the resulting χ_3^2 observation.

Solution. $x = 1.2^2 + (-0.5)^2 + 0.8^2 = 1.44 + 0.25 + 0.64 = 2.33$.

Interpretation. Each squared Normal contributes a non-negative amount, so χ_3^2 is supported on the positive half-line, as required by (5.4); the mean of χ_3^2 is 3, and this single observation of 2.33 is a plausible draw below that mean.

Example 3: Ratio method for t_5 .

Problem. Given $z = 1.5$ and an independent χ_5^2 draw $v = 6.0$, compute the resulting t_5 observation.

Solution.

$$t = \frac{z}{\sqrt{v/\nu}} = \frac{1.5}{\sqrt{6.0/5}} = \frac{1.5}{\sqrt{1.2}} = \frac{1.5}{1.0954} = 1.3693.$$

Interpretation. Because V is random, T has heavier tails than a standard Normal even though the numerator alone is standard Normal; this particular draw is only moderately larger in magnitude than a typical standard Normal value, consistent with $\nu = 5$ already being large enough that t_5 resembles the Normal reasonably closely (see the tail comparison figure above).

Example 4: Ratio method for $F_{3,5}$.

Problem. Given $u = 4.2$ (a χ_3^2 draw) and $v = 7.5$ (an independent χ_5^2 draw), compute the resulting $F_{3,5}$ observation.

Solution.

$$f = \frac{u/d_1}{v/d_2} = \frac{4.2/3}{7.5/5} = \frac{1.4}{1.5} = 0.9333.$$

Interpretation. Since $E(F_{3,5}) = d_2/(d_2 - 2) = 5/3 \approx 1.667$, this particular draw of 0.9333 lies below the theoretical mean, which is unsurprising given the right-skewness of the F distribution (most of its probability mass lies at moderate values, with a long right tail).

Applications

- **Statistics.** Chi-square, t , and F distributions are the reference distributions for goodness-of-fit tests, single-sample and two-sample mean comparisons, and the analysis of variance, respectively; simulation studies of the size and power of these tests require fast, exact generators for exactly these three distributions.
- **Machine Learning.** Gaussian mixture models, a standard tool for clustering and density estimation, are simulated directly by the composition method; compound and hierarchical generative models more broadly rely on the same two-stage logic.
- **Data Science.** Simulating synthetic datasets that mimic a heterogeneous population (e.g. customers belonging to distinct behavioural segments) is a direct application of the composition method.
- **Finance.** Regime-switching return models, in which market behaviour is drawn from one of several volatility regimes before returns are simulated, are compositions in exactly the sense of Section 5.2; F -type ratio statistics also appear in portfolio variance comparisons.
- **Reliability Engineering.** The convolution method generates Erlang (integer-shape Gamma) time-to-failure distributions as sums of independent Exponential component lifetimes in series reliability models; hyperexponential mixtures (composition method) model populations of components with heterogeneous failure rates.

- **Monte Carlo Methods.** Composition and convolution are frequently the fastest available exact generators, useful whenever a target distribution happens to admit either representation, avoiding the need for acceptance-rejection entirely.
- **Bayesian Statistics.** Hierarchical models — a prior on a parameter, followed by data generated conditional on that parameter — are, structurally, continuous-mixture (composition) models; prior predictive simulation is a direct application of Algorithm 5.1.
- **Operations Research.** Aggregate service or interarrival times built from sums of independent stage durations in multi-stage queueing networks are generated by the convolution method.
- **Engineering.** Simulating aggregate load or aggregate demand as a sum of independent component contributions is a direct convolution application.

Advantages

- When a valid distributional relationship is known, the resulting generator is typically simpler to implement and to verify than a general-purpose acceptance-rejection scheme, since no bounding constant or envelope density needs to be found.
- The methods are exact: the generated variate has precisely the target distribution, with no approximation, as proved in Theorems 5.1 and 5.2 and the derivations of Section 7.
- Composition and convolution generalize immediately to complicated hierarchical or compound structures that would be very difficult to describe by a single closed-form density, let alone invert or bound.

Limitations

- Not every distribution admits a convenient distributional relationship; when none is known, the general-purpose methods of Units 3.3 and 3.4 remain necessary.
- The convolution method for Chi-square, as implemented in Algorithm 5.2, has cost growing linearly in the degrees of freedom ν , making it inefficient for large ν relative to a direct Gamma-based generator, which also extends to non-integer degrees of freedom that the convolution method cannot reach at all.
- The composition method requires the mixture decomposition to be known in advance; discovering that an apparently complicated density is secretly a mixture is a modelling question, not something the simulation method itself can resolve.

Practical Tips

- **Generate all building blocks independently.** A very common implementation error is inadvertently reusing the same random draw for two supposedly independent components (for example, using the same uniform variate to select the mixture component and to generate within that component); this silently destroys the correctness guaranteed by Theorem 5.1.
- **Prefer vectorized generation over explicit loops in R.** As shown in `generate_chisq_convolution()`, drawing an entire matrix of Normal variates at once and summing rows is far faster in R than a loop generating one degree of freedom at a time.
- **Do not use the convolution method for large-parameter Chi-square or Gamma generation in production code.** Its computational cost scales linearly with the degrees of freedom, whereas dedicated Gamma generators (used internally by `rchisq()` and `rgamma()`) do not.
- **Verify mixture weights sum to one before simulating.** A silent normalization error in the component probabilities produces a mixture with the wrong proportions without raising any error.
- **When degrees of freedom are very small (e.g. $d_2 = 1$ or $d_2 = 2$ for the F distribution), check for numerical degeneracy.** Ratios of very small Chi-square draws can occasionally produce extreme values that are mathematically valid but numerically delicate in downstream calculations.

Summary

- Many named distributions are defined by, or provably equivalent to, explicit combinations of simpler random variables; exploiting this structure often gives faster and simpler generators than general-purpose methods.
- The composition method simulates a mixture distribution by first selecting a component (or parameter), then simulating within it; Theorem 5.1 shows the resulting marginal distribution is exactly the intended mixture.
- The convolution method simulates a sum of independent random variables directly; Theorem 5.2 (Gamma additivity) justifies generating χ_ν^2 as a sum of ν independent χ_1^2 variables.
- Student's t_ν and F_{d_1, d_2} are defined by explicit ratios of independent Normal and Chi-square variables; their densities (5.9) and (5.10) were derived directly from these ratio definitions using the same integral technique.
- These distributional-relationship generators are exact, but the convolution-based Chi-square generator in particular is computationally inferior to a direct Gamma-based generator for large degrees of freedom.

Key Formulae

- Composition (discrete mixture): $f(x) = \sum_j p_j f_j(x)$.
- Convolution: $X = Y_1 + \dots + Y_n$, Y_i independent.
- Gamma additivity: $\sum_i \text{Gamma}(\alpha_i, \beta) = \text{Gamma}(\sum_i \alpha_i, \beta)$.
- Chi-square: $\chi_\nu^2 = \sum_{i=1}^\nu Z_i^2$, $f(x) = x^{\nu/2-1} e^{-x/2} / (2^{\nu/2} \Gamma(\nu/2))$.
- Student's t : $T = Z / \sqrt{V/\nu}$, $f(t) = \frac{\Gamma((\nu+1)/2)}{\sqrt{\pi\nu} \Gamma(\nu/2)} (1 + t^2/\nu)^{-(\nu+1)/2}$.
- F distribution: $F = (U/d_1)/(V/d_2)$, $f(x) = \frac{\Gamma((d_1+d_2)/2)}{\Gamma(d_1/2)\Gamma(d_2/2)} (d_1/d_2)^{d_1/2} x^{d_1/2-1} (1 + d_1 x/d_2)^{-(d_1+d_2)/2}$.

Key Terms

- **Composition method.** Simulating from a mixture density by first selecting a component and then simulating within it.
- **Convolution.** The distribution of a sum of independent random variables.
- **Convolution method.** Simulating a target variable by generating and summing its independent constituent variables directly.
- **Distributional relationship.** A known probabilistic identity linking a target distribution to simpler, more easily simulated building blocks.
- **Mixture distribution.** A distribution arising from a two-stage random mechanism: select a component, then generate from within it.
- **Chi-square distribution.** The distribution of a sum of squared independent standard Normal variables.
- **Student's t distribution.** The distribution of a standard Normal variable divided by the square root of an independent, appropriately scaled Chi-square variable.
- **F distribution.** The distribution of the ratio of two independent, appropriately scaled Chi-square variables.

Exercises

Multiple Choice Questions

1. The composition method is most directly applicable when the target density can be written as
 - (a) a product of two densities
 - (b) a mixture of component densities
 - (c) a quantile function

- (d) a moment generating function
- 2. The convolution method generates a target variable as
 - (a) a ratio of independent variables
 - (b) a product of independent variables
 - (c) a sum of independent variables
 - (d) a maximum of independent variables
- 3. By definition, χ_ν^2 is the distribution of
 - (a) a single standard Normal variable
 - (b) the sum of ν independent squared standard Normal variables
 - (c) the ratio of two Normal variables
 - (d) the product of ν Uniform variables
- 4. Student's t_ν is defined as
 - (a) $Z \cdot \sqrt{V/\nu}$
 - (b) $Z/\sqrt{V/\nu}$
 - (c) Z^2/V
 - (d) V/Z
- 5. The F_{d_1, d_2} distribution is defined as
 - (a) $(U/d_1)/(V/d_2)$
 - (b) $(U \cdot d_1)/(V \cdot d_2)$
 - (c) $U + V$
 - (d) $U \cdot V$
- 6. Gamma additivity (Theorem 5.2) requires the summed Gamma variables to share
 - (a) a common shape parameter
 - (b) a common scale parameter
 - (c) a common mean
 - (d) no shared parameter at all
- 7. The convolution-based Chi-square generator has time complexity
 - (a) $O(n)$
 - (b) $O(n\nu)$
 - (c) $O(n^2)$
 - (d) $O(\log n)$
- 8. A common implementation error in the composition method is
 - (a) using too many components
 - (b) reusing the same random draw across independent stages
 - (c) normalizing the mixture weights
 - (d) using vectorized code
- 9. As degrees of freedom increase, the t_ν distribution
 - (a) diverges from the standard Normal
 - (b) converges toward the standard Normal
 - (c) becomes discrete
 - (d) becomes bimodal
- 10. The mean of F_{d_1, d_2} , when it exists, equals
 - (a) d_1/d_2
 - (b) $d_2/(d_2 - 2)$
 - (c) $d_1 \cdot d_2$
 - (d) 1

Answers. 1(b), 2(c), 3(b), 4(b), 5(a), 6(b), 7(b), 8(b), 9(b), 10(b).

Short Answer Questions

1. Explain, in your own words, why a distributional relationship can yield a faster generator than a general-purpose method.
2. State the composition method in two steps, using your own notation.

3. Why must the components summed in the convolution method be independent?
4. State the definition of the Chi-square distribution used in this unit, and explain why it is naturally suited to the convolution method.
5. Write down the ratio definition of Student's t_ν and identify the distribution of each of its two building blocks.
6. Write down the ratio definition of F_{d_1, d_2} and identify the distribution of each of its two building blocks.
7. Explain why the convolution-based Chi-square generator becomes computationally unattractive for large degrees of freedom.
8. Give one example, distinct from those in this unit, of a distribution well suited to the composition method.
9. Explain, intuitively, why the t distribution has heavier tails than the standard Normal.
10. Why does Theorem 5.2 require a *common* scale parameter across the summed Gamma variables?

Long Answer Questions

1. State and prove Theorem 5.1 (correctness of the composition method) for both the discrete and continuous mixture cases.
2. State and prove Theorem 5.2 (Gamma additivity) and explain how it is used to justify the Chi-square convolution generator.
3. Derive the density of χ_1^2 as the distribution of Z^2 for $Z \sim N(0, 1)$, showing every differentiation step.
4. Derive the density of Student's t_ν from its ratio definition, showing every integration step.
5. Derive the density of F_{d_1, d_2} from its ratio definition, showing every step of the bivariate transformation and integration.
6. Compare the composition, convolution, inverse transformation, and acceptance-rejection methods in terms of the information each requires about the target distribution.
7. Discuss the role of Chi-square, t , and F distributions in classical hypothesis testing, and explain why fast exact generators for these distributions matter for simulation studies of test performance.
8. Explain how the composition method underlies simulation from hierarchical Bayesian models, using a concrete two-level example of your choosing.
9. Discuss the computational trade-off between the convolution-based Chi-square generator and a direct Gamma-based generator, including how each behaves as the degrees of freedom parameter grows.
10. Explain, using the tail-comparison figure in this unit, how and why the t_ν distribution converges to the standard Normal distribution as $\nu \rightarrow \infty$.

Programming Exercises

1. Implement `generate_mixture()` from first principles (already provided) and reproduce the manual calculation of Example 1 by fixing the random seed appropriately.
2. Implement `generate_chisq_convolution()` from first principles (already provided) and verify, using a Kolmogorov-Smirnov test, that a sample of size 10,000 with $\nu = 6$ matches `rchisq()`.
3. Implement `generate_t_ratio()` and `generate_f_ratio()` from first principles (already provided) and reproduce Examples 3 and 4 by fixing the random seed appropriately.
4. Using `system.time()`, compare the speed of `generate_chisq_convolution()` against `rchisq()` for $\nu \in \{5, 50, 500\}$ and $n = 100,000$, and discuss the trend.
5. Write an R function implementing the composition method for a three-component Normal mixture with user-specified means, standard deviations, and weights, and produce a histogram overlaid with the theoretical mixture density.
6. Write an R function that simulates a Negative Binomial count as a sum of independent Geometric trial counts (the convolution method), and verify the result against `rnbinom()` using summary statistics.
7. Empirically verify Theorem 5.2 by simulating sums of independent Gamma variables with different shape parameters but a common scale, and comparing the sample distribution of the sum to the theoretical Gamma distribution using a Q-Q plot.
8. Write a function that simulates an Erlang (integer-shape Gamma) waiting time as a sum of independent Exponential interarrival times, and use it to estimate the probability that a five-stage service process

completes within a given time budget.

9. Reproduce the t -distribution tail-comparison figure of this unit for degrees of freedom $\nu \in \{1, 2, 5, 10, 30, 100\}$, and comment on the rate of convergence to the Normal density.
10. Write an R function that simulates from F_{d_1, d_2} using the ratio method and separately using R's built-in `rf()`, and compare their empirical densities using overlaid `ggplot2` density curves for several choices of (d_1, d_2) .

Mini Projects

1. **Building a compound reliability model.** Model the lifetime of a system composed of two sub-populations of components: a “weak” sub-population (Exponential with a high failure rate, representing early failures) and a “strong” sub-population (Exponential with a low failure rate), combined via the composition method with a chosen mixing weight. Simulate a large sample of system lifetimes, estimate the empirical hazard function, and discuss, with reference to your simulation, why a mixture of Exponentials can produce a hazard function that decreases over time even though each individual sub-population has a constant hazard rate.
2. **Simulation study of classical test statistics.** Using the Chi-square, t , and F generators developed in this unit, conduct a simulation study of the size (Type I error rate) of a one-sample t -test, a Chi-square goodness-of-fit test, and a one-way ANOVA F -test, each under its null hypothesis, across several sample sizes. Report the empirical rejection rate at the nominal 5% level for each test and sample size, and discuss any discrepancies between the nominal and empirical size.

Difficulty labels. Multiple choice, short answer: **Easy**. Long answer questions 3, 6, 7, 8, programming exercises 1-6: **Medium**. Long answer questions 1, 2, 4, 5, 9, 10, programming exercises 7-10: **Medium to Advanced**. Mini projects: **Advanced**.

References

- Casella, G., & Berger, R. L. (2002). *Statistical inference* (2nd ed.). Duxbury.
- Devroye, L. (1986). *Non-uniform random variate generation*. Springer-Verlag.
- Fisher, R. A. (1925). *Statistical methods for research workers*. Oliver and Boyd.
- Pearson, K. (1900). On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. *Philosophical Magazine*, 50(302), 157-175.
- Rizzo, M. L. (2019). *Statistical computing with R* (2nd ed.). CRC Press.
- Ross, S. M. (2020). *Simulation* (6th ed.). Academic Press.
- Student [Gosset, W. S.] (1908). The probable error of a mean. *Biometrika*, 6(1), 1-25.